I React to React Native Converter

MVP Architecture & Complete Project Documentation

Table of Contents

---

1. Introduction

Project Name

AI React-to-React Native Component Converter

Objective

Is project ka objective ek aisa AI-powered tool banana hai jo React web component ko React Native component me automatically convert kar sake bina UI ko break kiye.

Main Goals

- Same UI preserve karna
- Same spacing maintain karna
- Same layout maintain karna
- Same styling preserve karna
- Responsive design maintain karna
- Manual conversion effort reduce karna

---

2. Problem Statement

React aur React Native internally different rendering systems use karte hain.

React Web

- HTML elements
- Browser DOM
- CSS engine
- Browser events

React Native

- Native Views
- Native rendering
- StyleSheet API
- Mobile gesture system

Problem

Direct conversion se:

- UI break ho sakta hai
- Styling mismatch ho sakti hai
- Layout distort ho sakta hai
- Unsupported components aa sakte hain

Isliye hybrid AI + AST + Visual Validation architecture required hai.

---

3. Project Vision

Long-term vision:

- One-click React → React Native conversion
- Production-ready mobile UI generation
- 90%+ visual similarity
- AI-assisted repair system
- Enterprise-grade conversion engine

---

4. MVP Goals

MVP Scope

Initial MVP me sirf:

- Single component conversion
- Functional components
- Tailwind support
- Basic CSS support
- Flex layouts
- Buttons
- Forms
- Cards
- Text components

support kiya jayega.

---

5. Core Features

Main Features

1. React Component Upload

User React component upload karega.

2. AST Parsing

System component structure analyze karega.

3. JSX Mapping

Web elements ko React Native elements me convert karega.

4. Style Conversion

CSS ko RN-compatible styles me convert karega.

5. AI Repair Engine

Unsupported cases ko fix karega.

6. Screenshot Comparison

UI similarity detect karega.

7. Auto Repair Loop

Visual mismatch automatically fix karega.

8. Final Export

Optimized React Native component generate karega.

---

6. Complete MVP Workflow

React Component
↓
AST Parser
↓
JSX Analyzer
↓
Component Mapper
↓
Style Extraction
↓
Tailwind Conversion
↓
RN Code Generator

↓
AI Repair Pass
↓
React Screenshot
↓
RN Screenshot
↓
Pixel Comparison
↓
Mismatch Detection
↓
AI Auto Fix
↓
Final RN Component

---

7. System Architecture

High-Level Architecture

Frontend Dashboard
↓
API Gateway
↓
Conversion Engine
↓
AI Engine
↓
Visual Testing Engine
↓
Database

---

8. Frontend Architecture

Responsibilities

- File upload
- Code editor
- Live preview
- Conversion status
- Error logs
- Screenshot comparison

- Final export

Recommended Frontend Stack

Tool	Reason
React	Fast component-based UI
Monaco Editor	VS Code-like editing
Tailwind CSS	Faster UI styling
Zustand	Lightweight state management

---

9. Backend Architecture

Responsibilities

- File processing
- AST conversion
- AI requests
- Screenshot automation
- Queue processing
- Result storage

Recommended Backend Stack

Tool	Reason
Node.js	JS ecosystem compatibility
Express.js	Lightweight APIs
BullMQ	Queue management
Redis	Fast caching & queues

---

10. AI Engine Architecture

AI Responsibilities

- Unsupported syntax handling
- Semantic analysis
- Layout correction
- Style fixing
- Animation migration
- Error repair

Recommended AI Models

AI Model	Reason
GPT-5	Strong reasoning
Claude Opus	Large file support
Gemini Pro	Large context handling

---

11. AST Parsing System

Why AST Important Hai

Direct text replacement unreliable hota hai.

AST parsing:

- Structure samajhta hai
- JSX analyze karta hai
- Imports detect karta hai
- Styles extract karta hai
- Hooks identify karta hai

Recommended Tools

Babel Parser

Use:

- JSX parsing
- TypeScript parsing

Reason:

- Industry standard

Recast

Use:

- AST transformation
- Code regeneration

Reason:

- Original formatting preserve karta hai

ts-morph

Use:

- Deep TypeScript analysis

Reason:

- Advanced AST control

---

12. Style Conversion System

Main Challenge

React Native CSS directly support nahi karta.

Solution Pipeline

CSS
↓
Parsed CSS AST
↓
Tailwind Mapping
↓
NativeWind Classes
↓
RN Styles

Recommended Tools

Tool	Reason

PostCSS	CSS parsing
CSSTree	Advanced CSS analysis
NativeWind	Tailwind support in RN

---

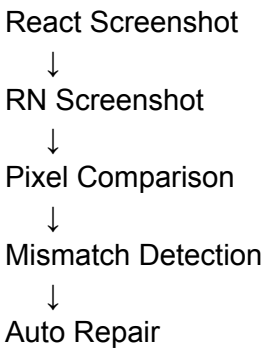
13. Visual Matching System

Why Needed

Sirf code conversion enough nahi hai.

Visual similarity required hai.

Workflow



Recommended Tools

Tool	Reason
Playwright	Browser screenshots
Detox	RN app testing
Pixelmatch	Pixel-perfect comparison

---

14. Database Design

Tables

Users

- id
- email
- created_at

Projects

- id
- user_id
- project_name
- created_at

Conversions

- id
- project_id
- input_code
- output_code
- screenshots
- similarity_score

Logs

- id
- conversion_id
- error_logs
- AI_repairs

Recommended Database

PostgreSQL

Reason:

- Reliable relational database
- Scalable architecture

---

15. APIs Structure

API	Method	Purpose
/upload	POST	Upload component

/convert	POST	Start conversion
/screenshot	POST	Generate screenshots
/compare	POST	Compare UI
/export	GET	Export final code

---

16. Folder Structure

frontend/
backend/
parser/
ai-engine/
visual-engine/
shared/
configs/
docker/

---

17. Technology Stack

Frontend

- React
- Tailwind
- Monaco Editor
- Zustand

Backend

- Node.js
- Express.js
- BullMQ
- Redis

AI

- GPT-5
- Claude
- Gemini

Parsing

- Babel Parser
- Recast
- ts-morph

Styling

- PostCSS
- NativeWind

Testing

- Playwright
- Detox
- Pixelmatch

Database

- PostgreSQL

Deployment

- Docker
- Nginx

---

18. Tools & Reasons

Tool	Why Use It
Babel Parser	JSX parsing
Recast	AST transformation
ts-morph	TS analysis
PostCSS	CSS parsing
NativeWind	RN Tailwind support
GPT-5	AI reasoning

Playwright	Screenshot testing
Pixelmatch	UI comparison
Detox	RN testing
Docker	Containerization
PostgreSQL	Data storage

---

19. AI Prompt Engineering

Prompt Example

Convert the following React component into React Native while preserving:

- Same layout
- Same spacing
- Same colors
- Same hierarchy
- Same responsive behavior

Rules:
- Use NativeWind
- Avoid unsupported APIs
- Replace HTML tags with RN components
- Maintain visual similarity

---

20. Conversion Pipeline

Input React File
↓
AST Parsing
↓
Component Detection
↓
Tag Mapping
↓
Style Parsing
↓
Tailwind Translation
↓
RN Generation

↓
AI Repair
↓
Visual Validation
↓
Final Output

---

21. Auto Repair Loop

Logic

Agar visual difference threshold se zyada ho:

Analyze mismatch
↓
Send repair prompt to AI
↓
Regenerate styles
↓
Retest screenshots
↓
Repeat until acceptable similarity

---

22. Deployment Architecture

Recommended Deployment

Frontend

- Vercel

Backend

- AWS EC2
- Docker containers

Database

- PostgreSQL Cloud

Queue System

- Redis Cloud

---

23. Security Considerations

Security Features

- File validation
- Sandbox execution
- Docker isolation
- Rate limiting
- API authentication
- Malware scanning

---

24. Scalability Plan

Future Scaling

- Microservices architecture
- Distributed workers
- GPU inference
- Multi-region deployment
- AI caching

---

25. Future Scope

Future Features

- Full project conversion
- Figma integration
- Drag-and-drop editor
- Real-time preview
- Animation conversion
- Design token extraction
- AI design optimization

---

26. Development Roadmap

Phase 1

- React parsing
- Basic JSX mapping
- RN generation

Phase 2

- Tailwind support
- NativeWind integration
- Style conversion

Phase 3

- AI repair system
- Screenshot comparison
- Auto repair loop

Phase 4

- Cloud deployment
- Multi-user support
- Export system

---

27. Conclusion

Ye project ek advanced AI engineering platform hoga jo:

- AST parsing
- AI reasoning
- Screenshot validation
- Visual repair systems
- React Native generation

sab combine karega.

Main Advantages

- High UI accuracy
- Reduced manual work
- Scalable architecture
- AI-assisted correction
- Production-ready workflow

Agar proper architecture follow kiya jaye to ye tool enterprise-level React-to-React Native conversion platform ban sakta hai.